



Name: ..... Cohort/Batch: ..... Date: ..... Score: .....

## ISTIO PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Kubernetes

TOTAL: 10 QUESTIONS

**Q1** What problem does Istio solve in DevOps or infrastructure? Infrastructure Model

---

---

**Q6** How does Istio integrate with CI/CD pipelines? Observability

---

---

**Q2** What is Istio's core architecture? Architecture

---

---

**Q7** How do you debug a failed Istio deployment? Lifecycle

---

---

**Q3** How does Istio define infrastructure or configuration as code? Configuration

---

---

**Q8** What are security best practices for Istio? Compliance

---

---

**Q4** How does Istio ensure idempotency? Hardening

---

---

**Q9** How does Istio scale for large environments? Testing

---

---

**Q5** How do you handle secrets in Istio? SSO / IdP

---

---

**Q10** What are common mistakes teams make when adopting Istio? Adoption

---

---



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

**A1 Istio addresses a specific concern provisioning,** Mitigation  
Istio addresses a specific concern provisioning, configuration management, orchestration, or CI/CD by automating manual steps that are error-prone, slow, or not reproducible when done by hand.

**A6 CI/CD pipelines call Istio in automated steps:** Observability  
CI/CD pipelines call Istio in automated steps: plan on a pull request to preview changes and apply on merge to main. Approval gates can require a human to review the plan before changes go to production.

**A2 Istio has a control plane that stores** Primitives  
Istio has a control plane that stores desired state and a data plane (agents or push-based SSH) that enforces it. Understanding this distinction clarifies where configuration is evaluated and where changes are applied.

**A7 Check Istio's logs for the specific error** Automation  
Check Istio's logs for the specific error message and the resource that failed. For infrastructure tools, re-run with increased verbosity (-v, --debug). Review the state file or event history to understand what changed before the failure.

**A3 Istio uses declarative files (YAML, HCL, or** Hardening  
Istio uses declarative files (YAML, HCL, or a DSL) to express desired state. A plan/diff phase shows what will change before applying. Version-controlling these files enables peer review and rollback.

**A8 Rotate credentials used by Istio, restrict network** Governance  
Rotate credentials used by Istio, restrict network access to its control plane, enable audit logging of all operations, and use least-privilege service accounts. Never run Istio with admin credentials in production.

**A4 Istio operations are designed to produce the** Gotchas  
Istio operations are designed to produce the same result when run multiple times. Applying the same config twice converges to the desired state rather than creating duplicate resources. This makes automation safe to re-run.

**A9 Large Istio deployments use workspaces or namespaces** Assessment  
Large Istio deployments use workspaces or namespaces to isolate environments, remote state backends for team collaboration, and modular code to avoid a single monolithic config that is slow to plan/apply.

**A5 Secrets should never be stored in plain** Federation  
Secrets should never be stored in plain text in Istio config files. Use a secrets backend (Vault, AWS Secrets Manager) and inject values at runtime via environment variables or encrypted vaults supported by the tool.

**A10 Common pitfalls include storing secrets in plain** Optimization  
Common pitfalls include storing secrets in plain text config, skipping the plan step before apply, not reviewing state drift, and not having a rollback strategy when a deployment fails partway through.